

Information Retrieval #5 – Index Construction

Severin Mills

20.03.2026



Inhaltsverzeichnis/Agenda

1. Lecture Recap
2. Exercise 4: Index Construction
3. Exam Questions
4. (Kahoot)

B+-Tree Visualization

Visualization by Prof. Dr. Galles

www.cs.usfca.edu/~galles/visualization/BPlusTree.html

Definition might differ from what we defined in the lecture though!

Term Conversion

Currently:

ETH	1
computer	1
data	1
information	1

CPU	2
ETH	2
information	2
computer	2

retrieval	7
Zürich	7

Zürich	3
ETH	3
CPU	3
data	3
retrieval	3

Zürich	4
CPU	4
information	4
computer	4

data	5
retrieval	5

Term Conversion

Currently:

Waste of Space!

=> Use termIDs

ETH	1
computer	1
data	1
information	1

CPU	2
ETH	2
information	2
computer	2

retrieval	7
Zürich	7

Zürich	3
ETH	3
CPU	3
data	3
retrieval	3

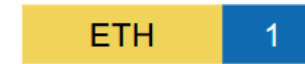
Zürich	4
CPU	4
information	4
computer	4

data	5
retrieval	5

Term Conversion

Use term-termID mapping!

token in memory



let's take 10 bytes in average 4 bytes

$$14 \text{ bytes} \times 100,000,000 = 1.4 \text{ GB}$$



4 bytes (always) 4 byte

$$8 \text{ bytes} \times 100,000,000 = 0.8 \text{ GB}$$

Constructing the index

We want to optimize

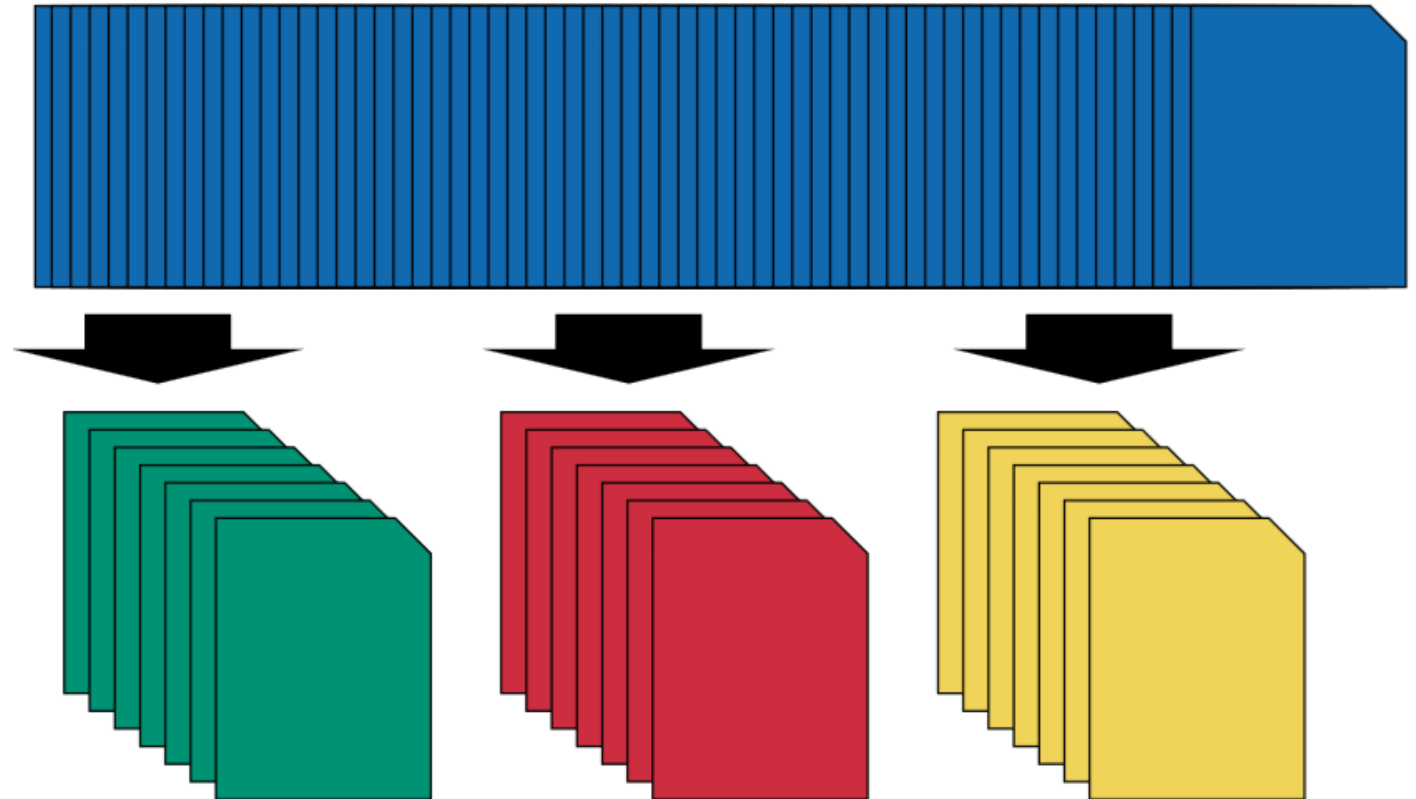
- Capacity (we want high)
- Latency (we want low)
- Throughput (we want high)

Use RAM for most of the work. Try to have few requests to disk.

Blocked Sort-Based Indexing (BSBI)

1. Shard the collection of documents
(i.e. split them up into blocks)

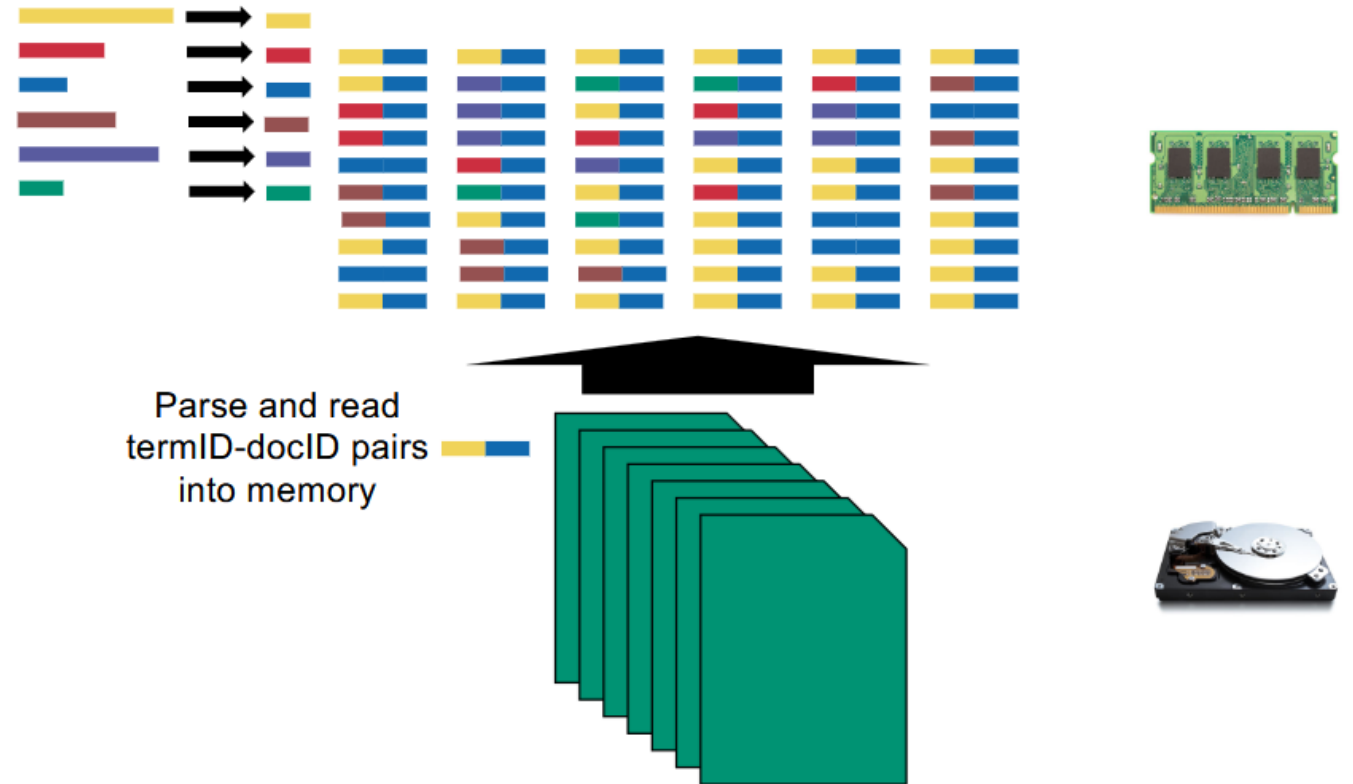
=> i.e. Batch processing



Blocked Sort-Based Indexing (BSBI)

2. Process each block one by one in memory

- **Parse term-termID pairs**
- Sort pairs according to termID
- Write back intermediate results



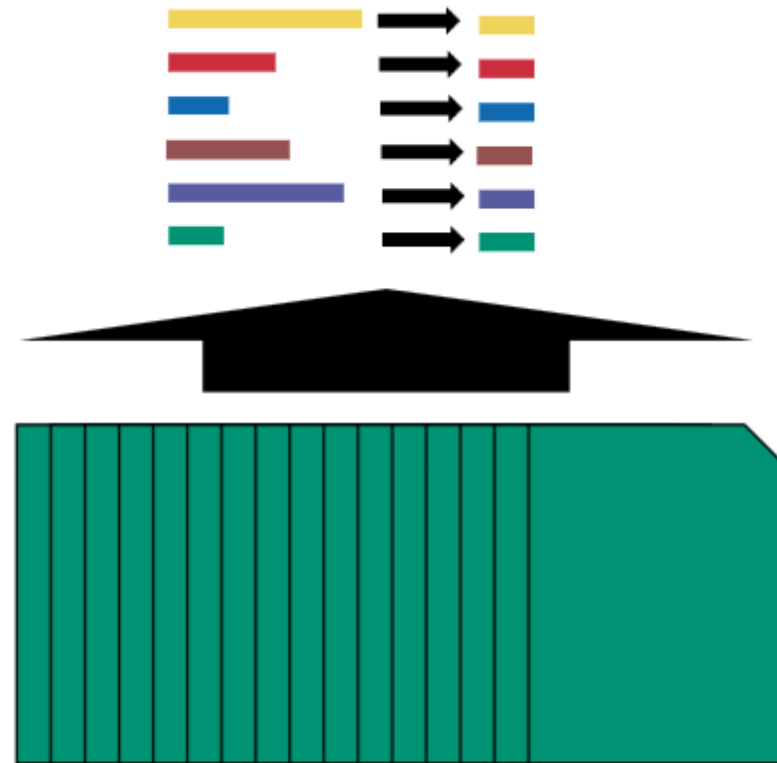
Generating termIDs

On the fly:

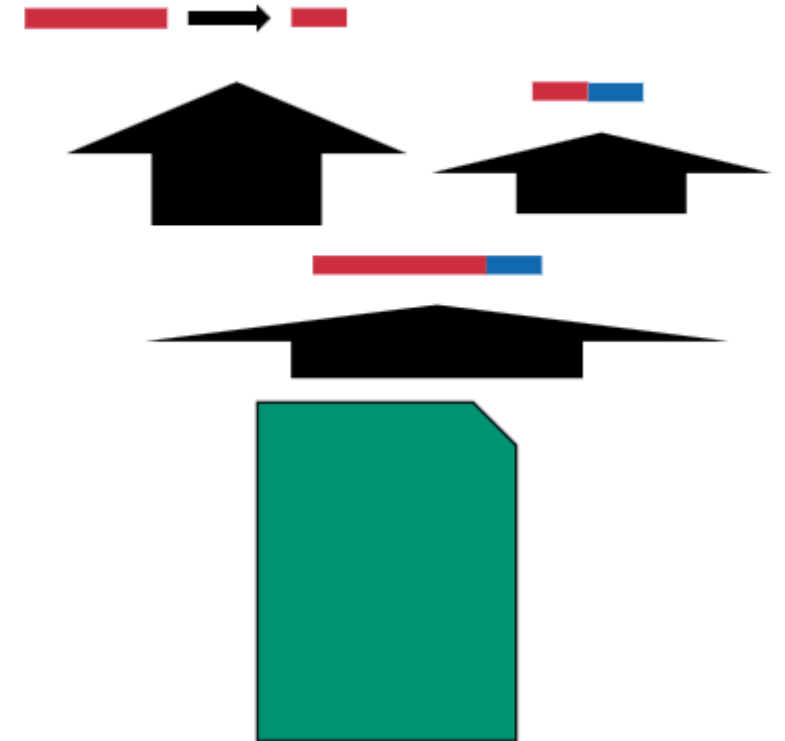
- + single pass over corpus
- IDs are arbitrary, no locality, no sorting

First pass:

- + vocabulary is fully known before processing
- + IDs can be sorted alphabetically
- two passes over corpus (expensive)



Strategy 1
Do a first pass to
build term-termID mapping

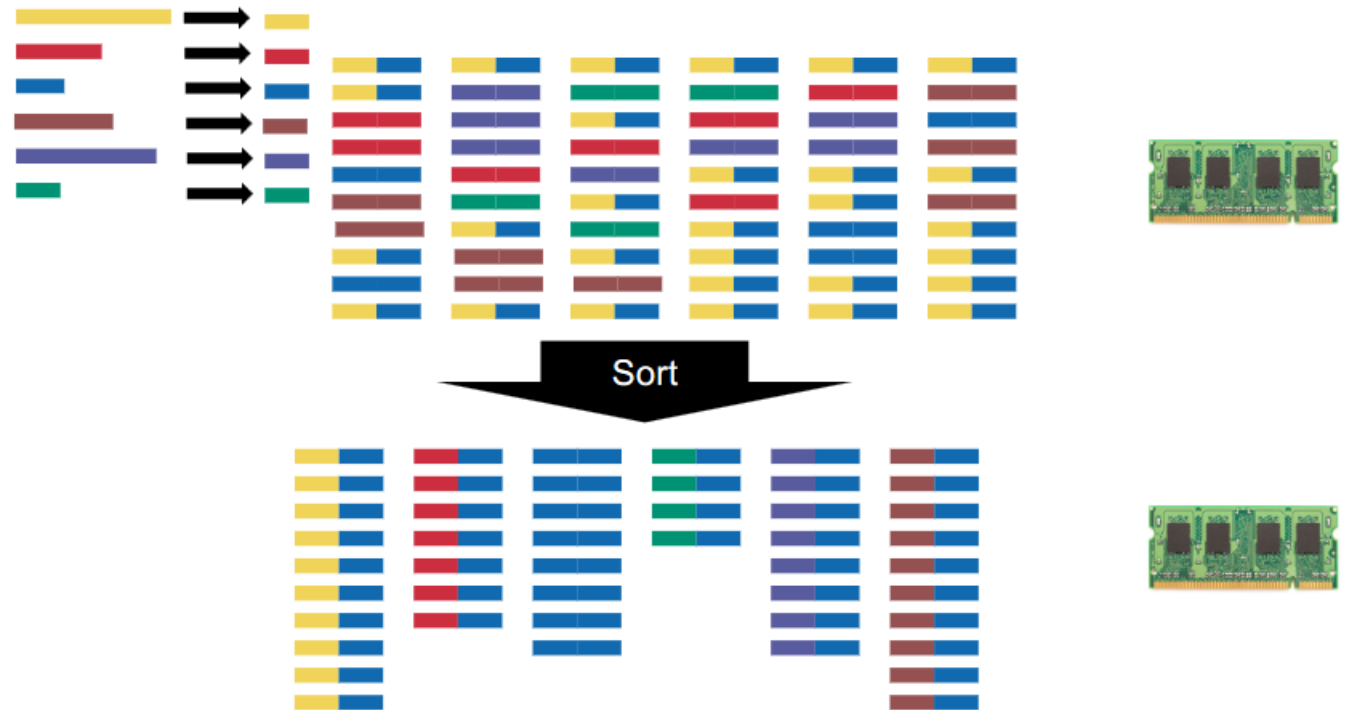


Strategy 2
On the fly

Blocked Sort-Based Indexing (BSBI)

2. Process each block one by one in memory

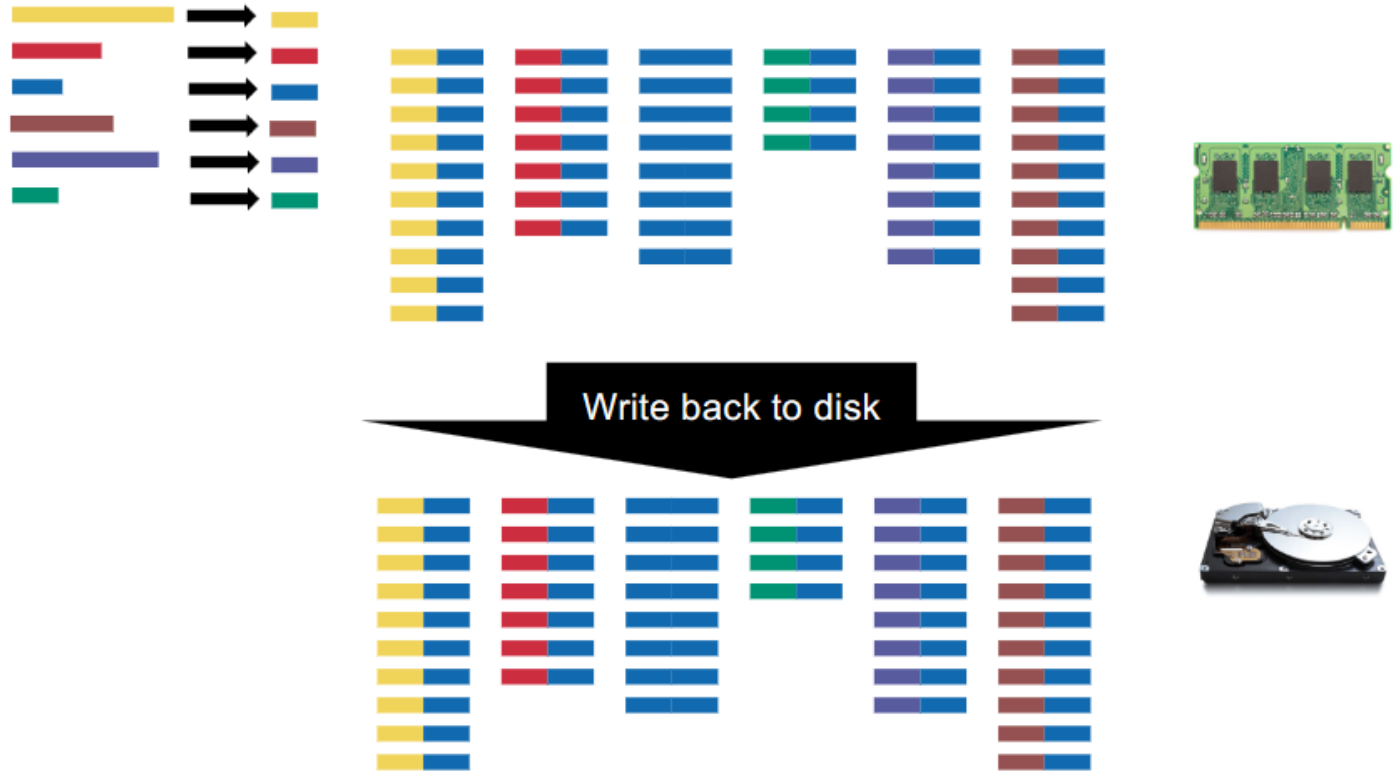
- Parse term-termID pairs
- **Sort pairs according to termID**
- Write back intermediate results



Blocked Sort-Based Indexing (BSBI)

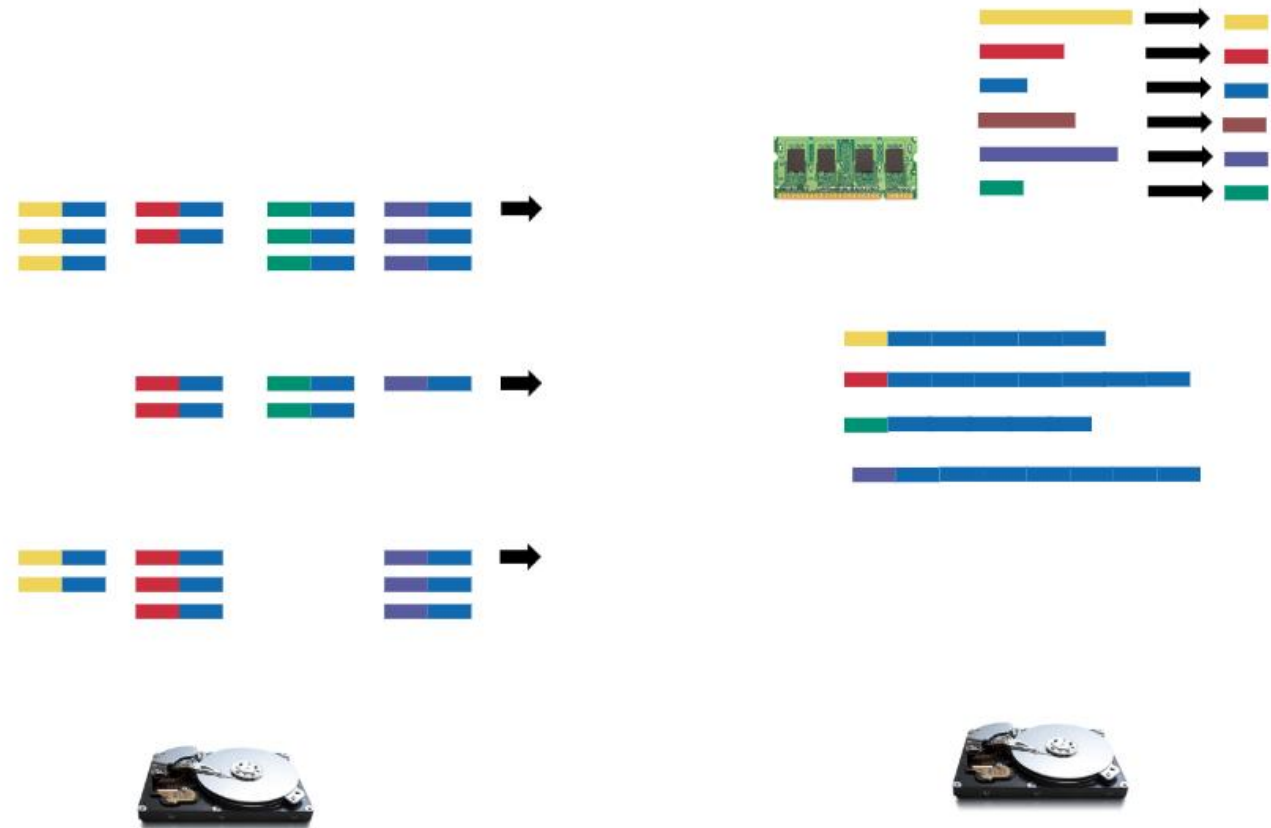
2. Process each block one by one in memory

- Parse term-termID pairs
- Sort pairs according to termID
- **Write back intermediate results**



Blocked Sort-Based Indexing (BSBI)

3. Merge intermediate results into the final index



Blocked Sort-Based Indexing (BSBI)

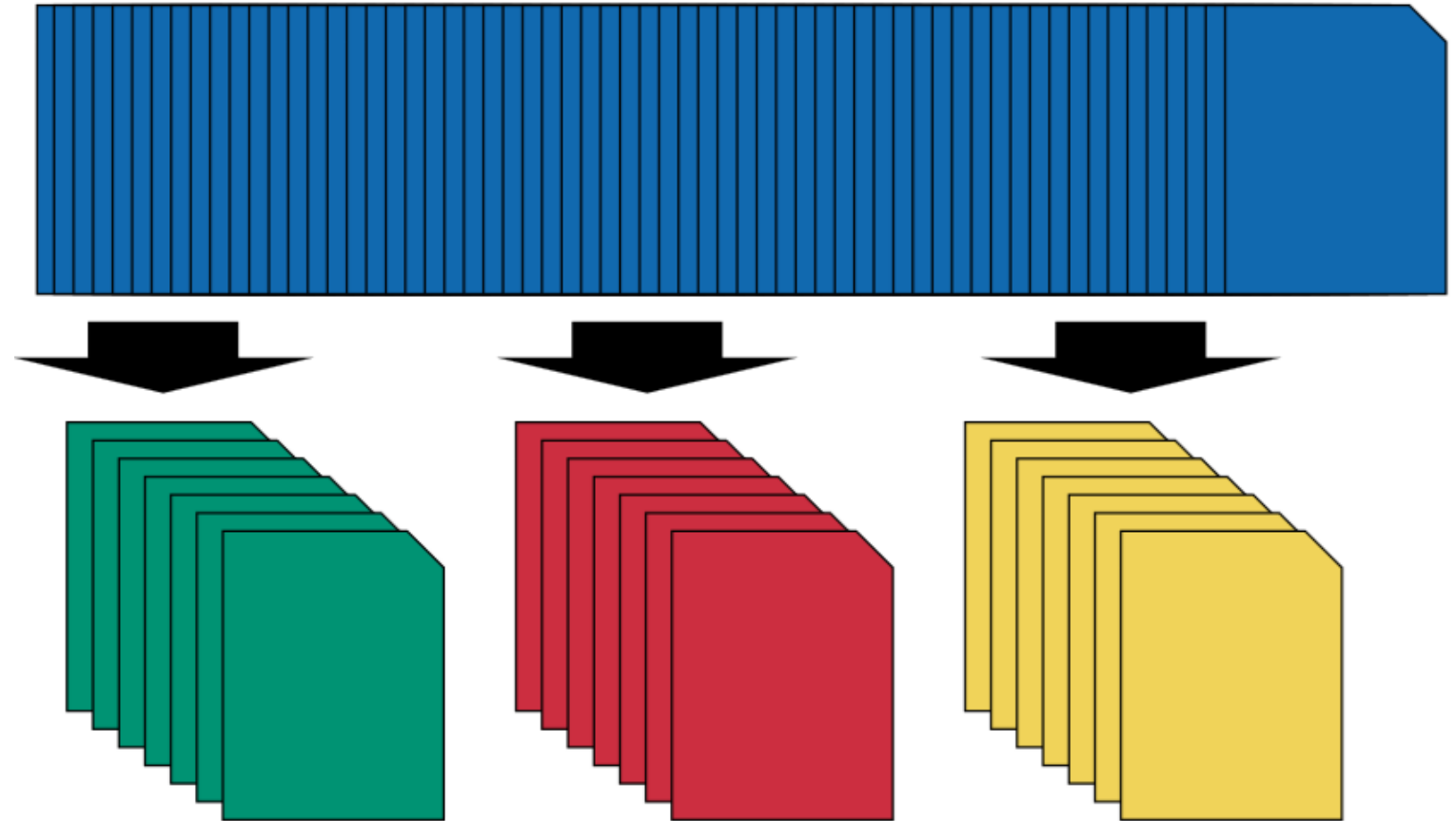
Complexity: $O(T \log T)$

- First and second step (sorting): $O(T \log T)$
- Third step (merging): $O(T)$

$T = \text{\#tokens}$

Single-Pass In-Memory Indexing (SPIMI)

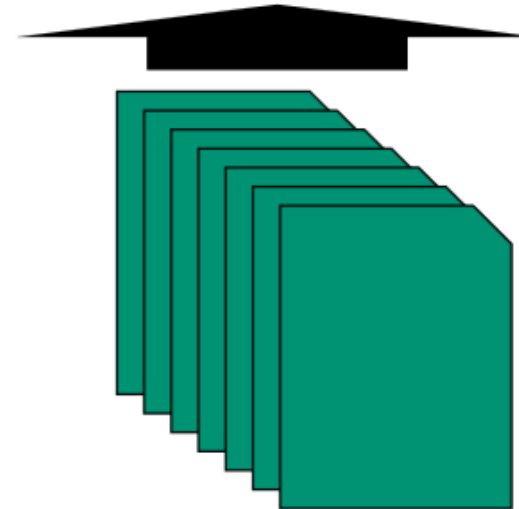
1. Shard collection of documents into blocks



Single-Pass In-Memory Indexing (SPIMI)

2. Process each block

- **Parse term-docID pairs**
- **Create a dictionary**
- Sort on terms
- Write back intermediate results



Single-Pass In-Memory Indexing (SPIMI)

2. Process each block

- Parse term-docID pairs
- Create a dictionary
- **Sort on terms**
- Write back intermediate results



Single-Pass In-Memory Indexing (SPIMI)

2. Process each block

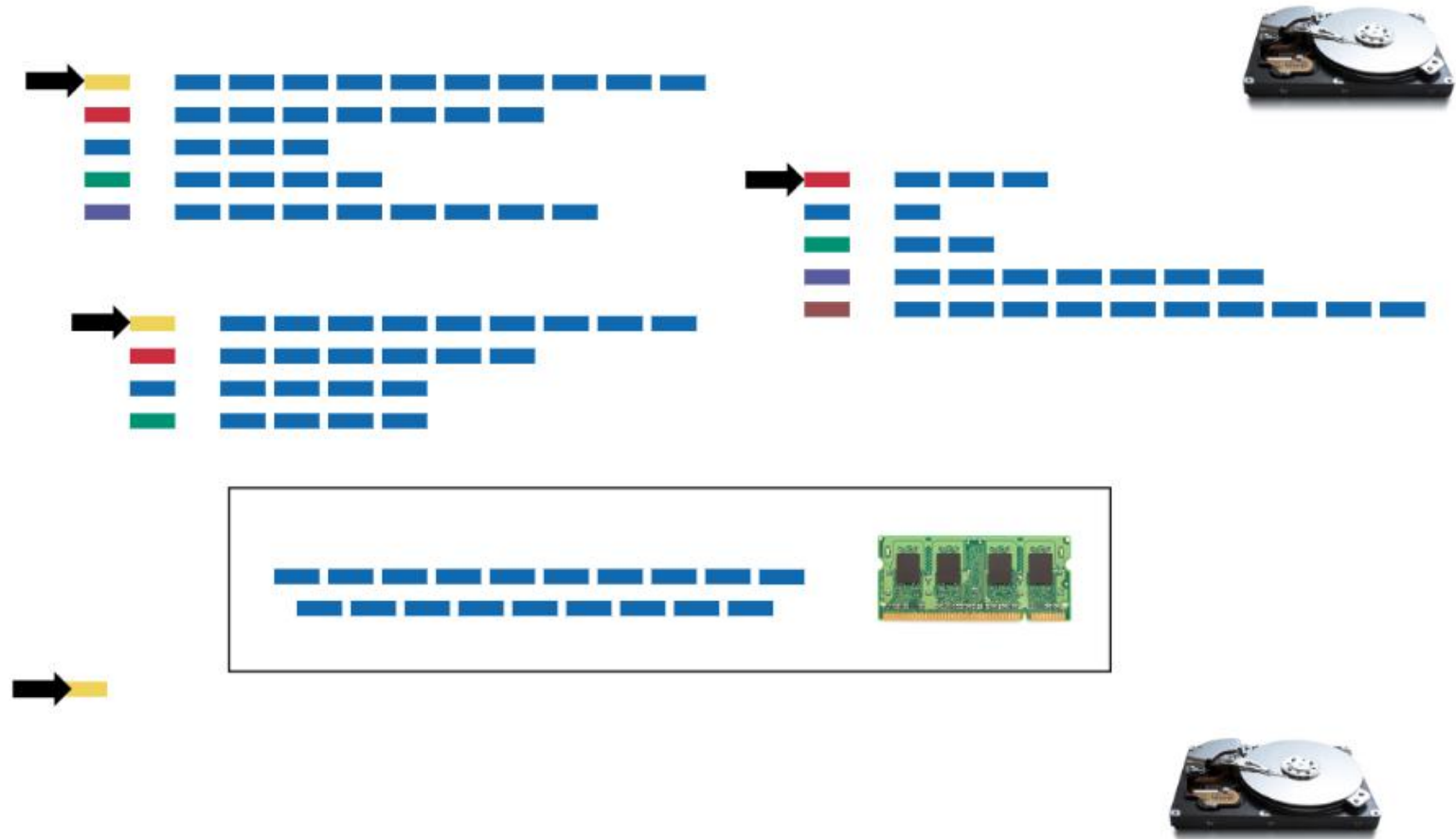
- Parse term-docID pairs
- Create a dictionary
- Sort on terms
- **Write back intermediate results**

=> No term-termID mapping in memory!



Single-Pass In-Memory Indexing (SPIMI)

3. Final merge



Single-Pass In-Memory Indexing (SPIMI)

Complexity: $O(T)$

- First and second step (processing): $O(T \log M)$
- Third step (merging): $O(T)$

$T = \text{\#tokens}$, $M = \text{\#terms}$, $M \ll T$

What is the difference?

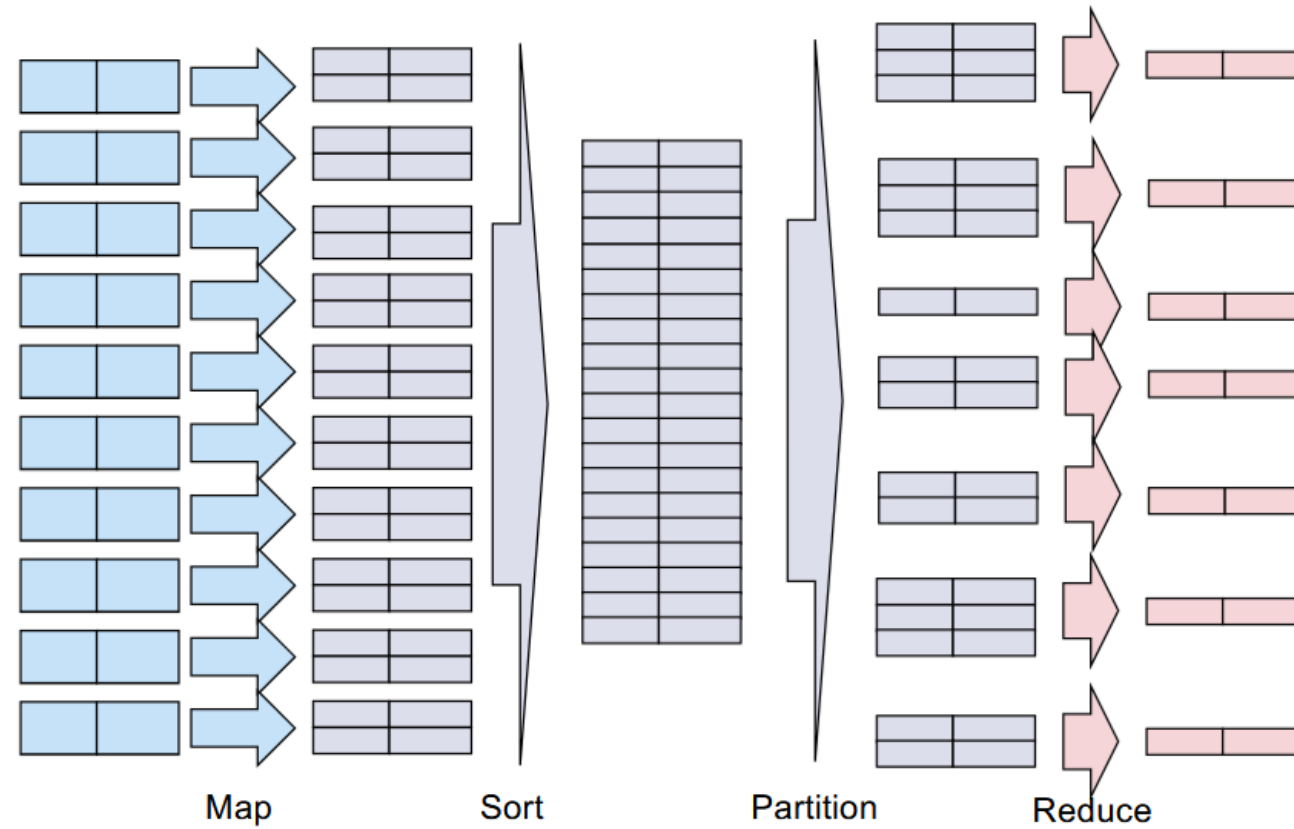
BSBI

- Keeps term-termID mapping in memory
- First pass or “on the fly” for collecting mappings
- Merges to postings list in disk at the end

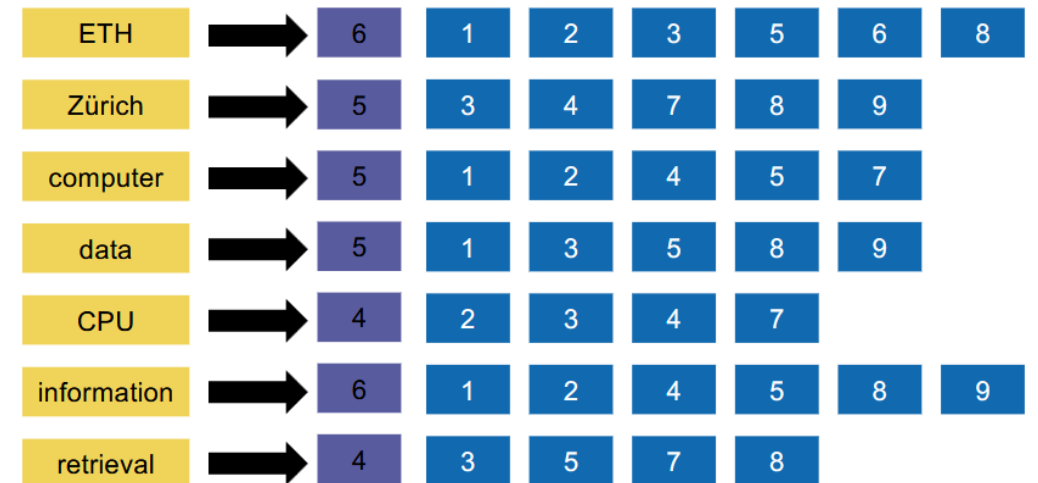
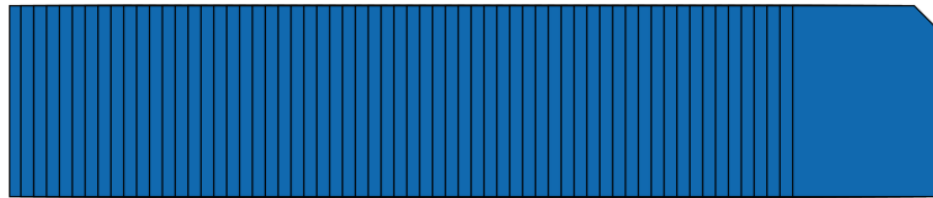
SPIMI

- Does not need a term-termID mapping
- Adds postings directly to intermediate postings list
- More scalable since not limited by memory size
- Uses less memory, is faster

MapReduce

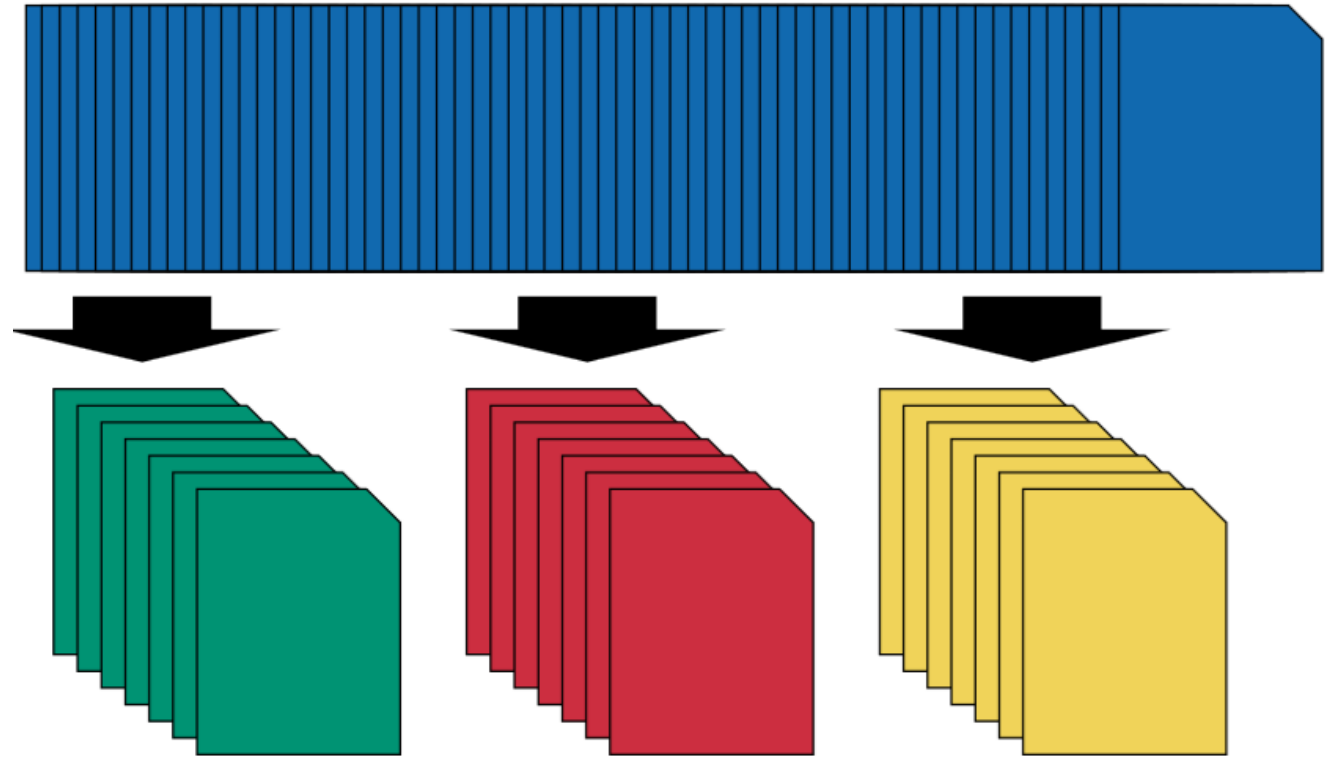


Case Study: MapReduce



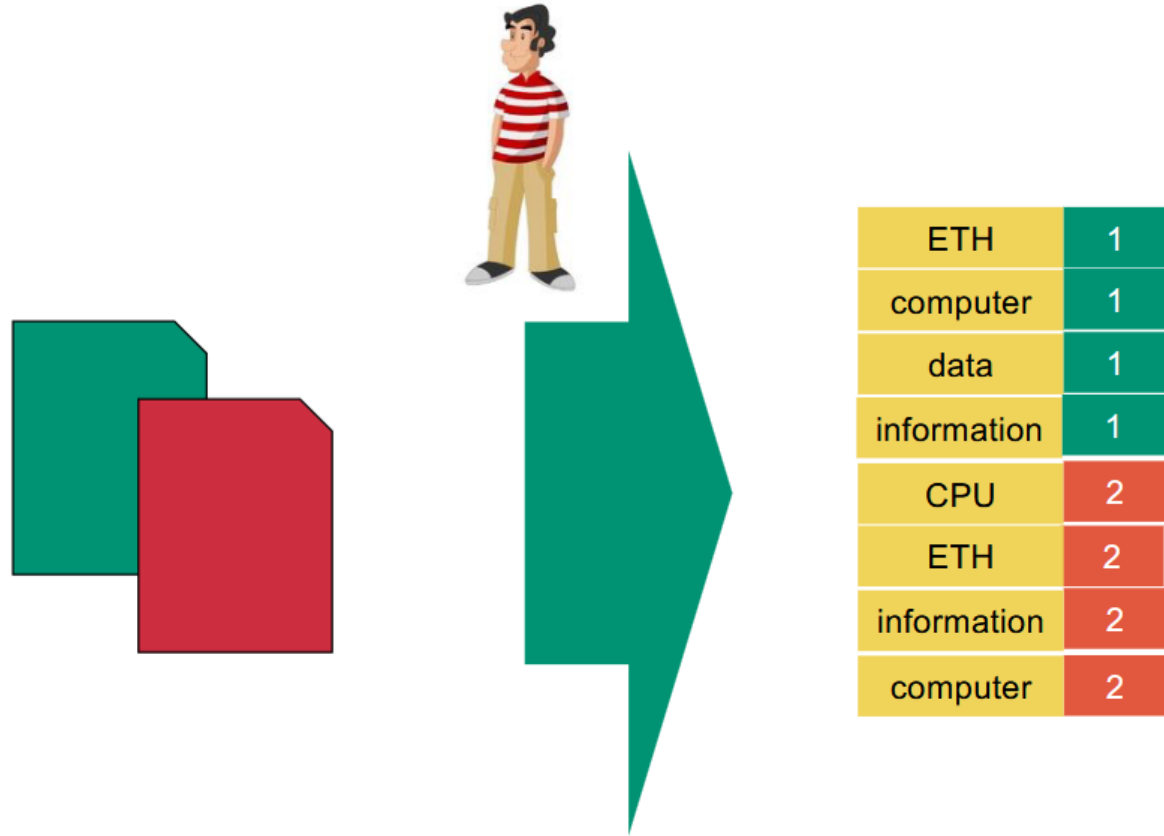
Case Study: MapReduce

1. Shard the collection



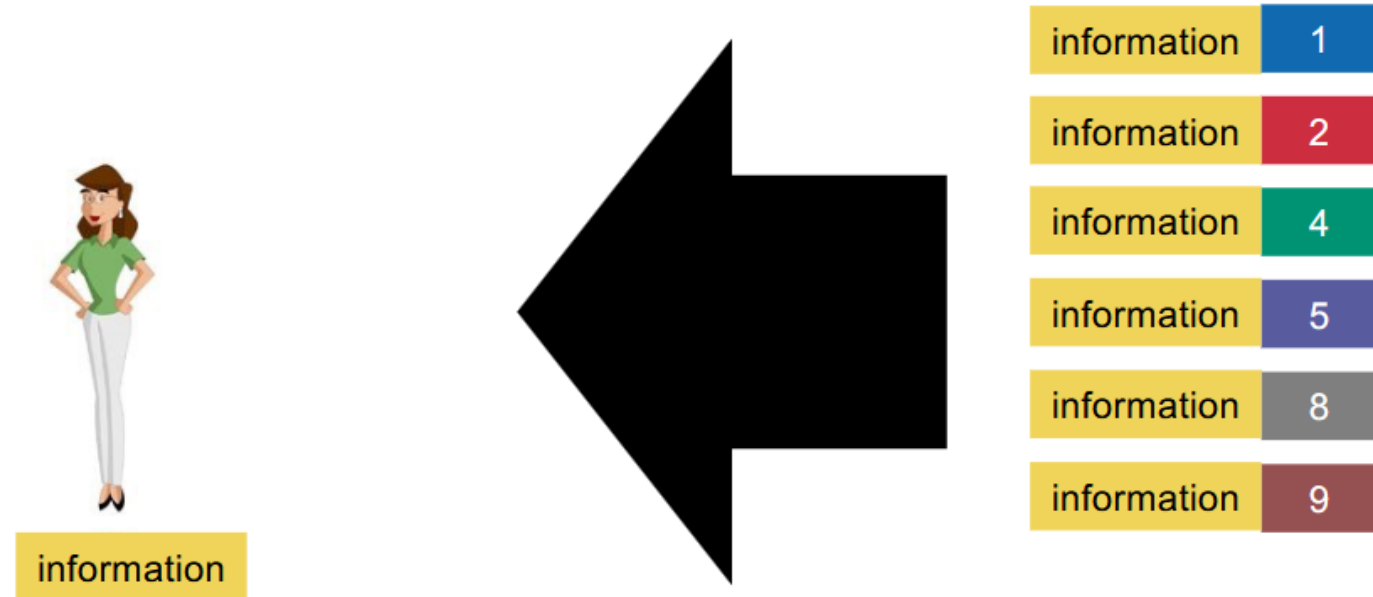
Case Study: MapReduce

Mappers



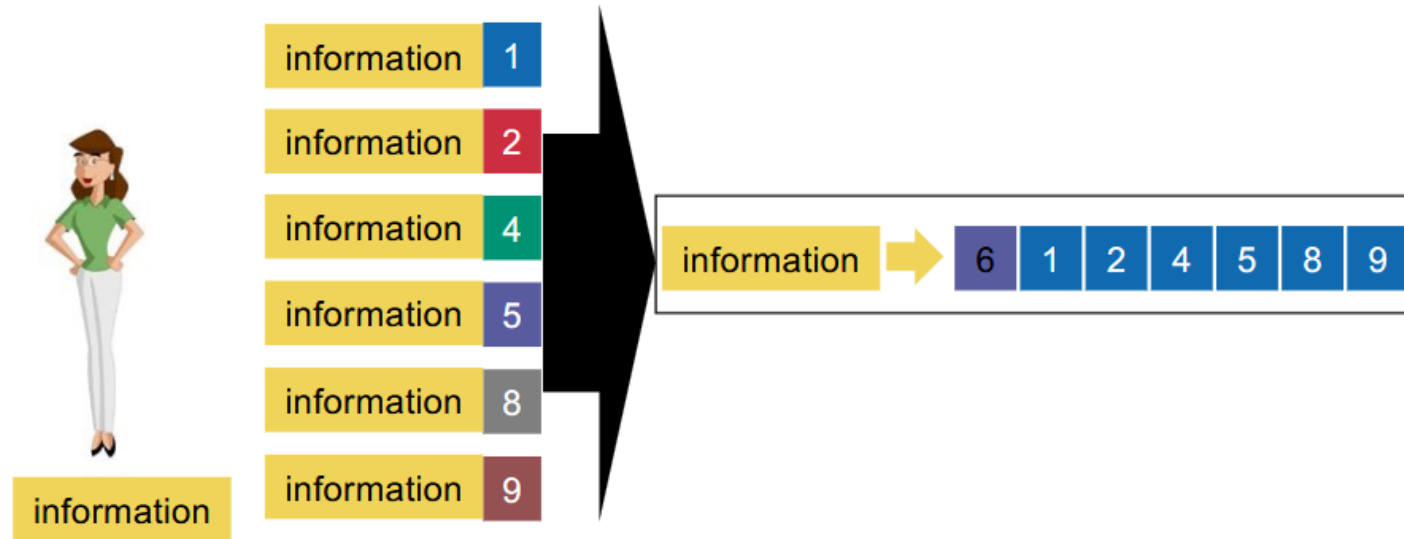
Case Study: MapReduce

Reducers



Case Study: MapReduce

Reducers



Case Study: MapReduce

End result:

ETH	→	6	1	2	3	5	6	8
Zürich	→	5	3	4	7	8	9	
computer	→	5	1	2	4	5	7	
data	→	5	1	3	5	8	9	
CPU	→	4	2	3	4	7		
information	→	6	1	2	4	5	8	9
retrieval	→	4	3	5	7	8		

Updating an index

Reality: Large document collections are usually not static => documents being updates, deleted, added

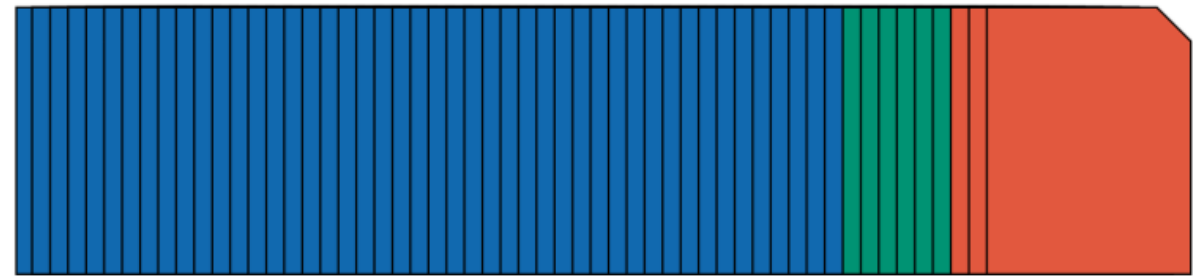
Two ways:

- Periodic reconstruction
- Auxiliary index

Periodic Reconstruction

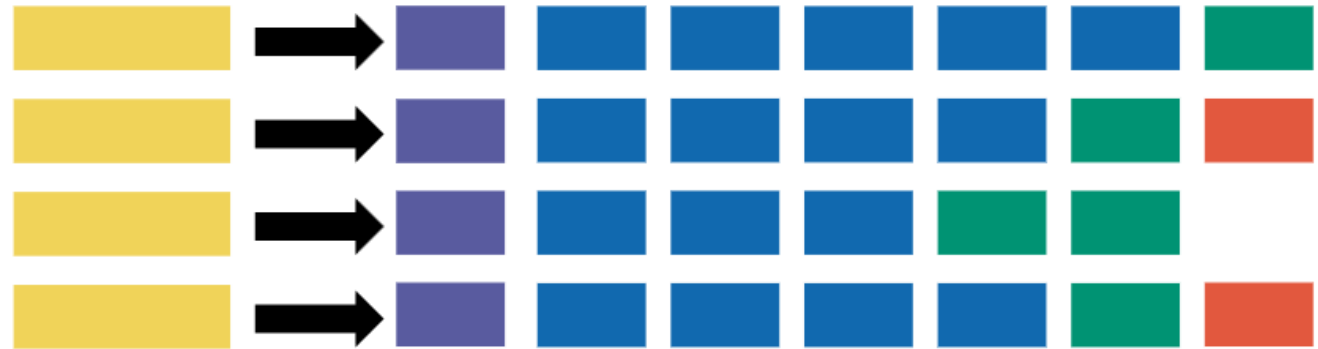
Problems:

- Result staleness
- Potentially unavailable or slower during reconstruction



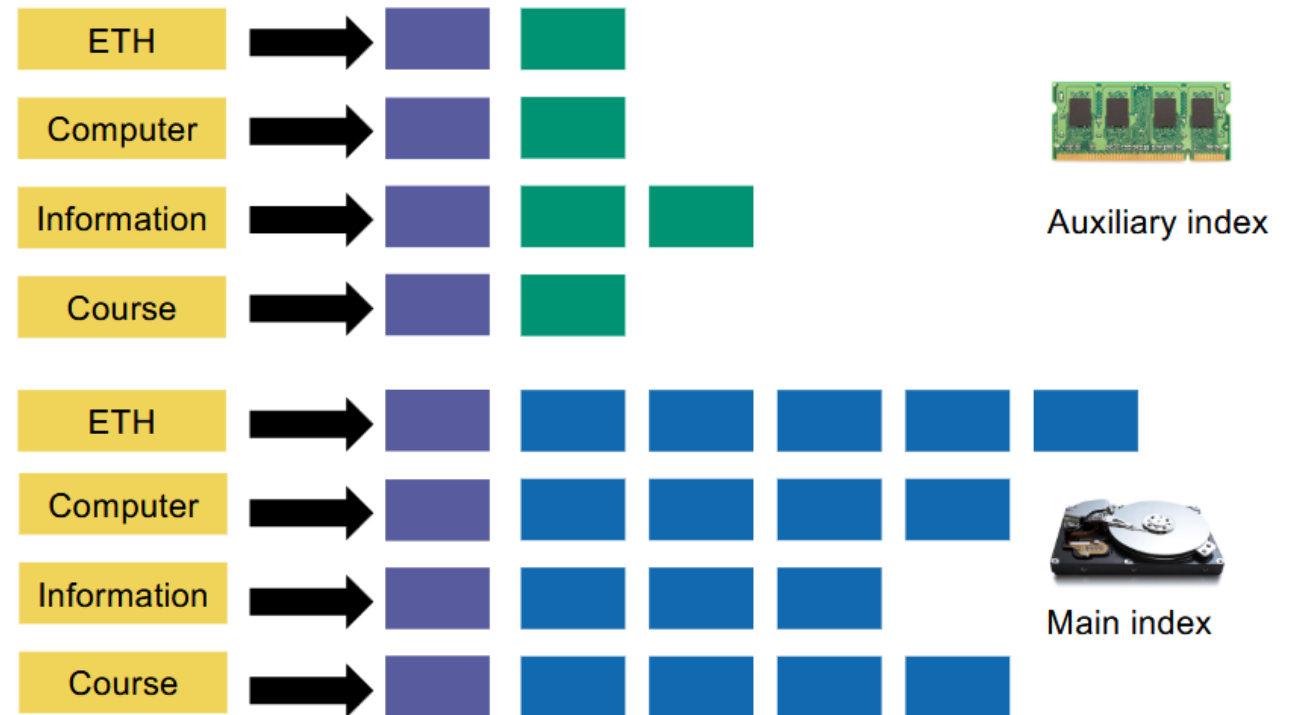
Reconstruct

Still used in practice.



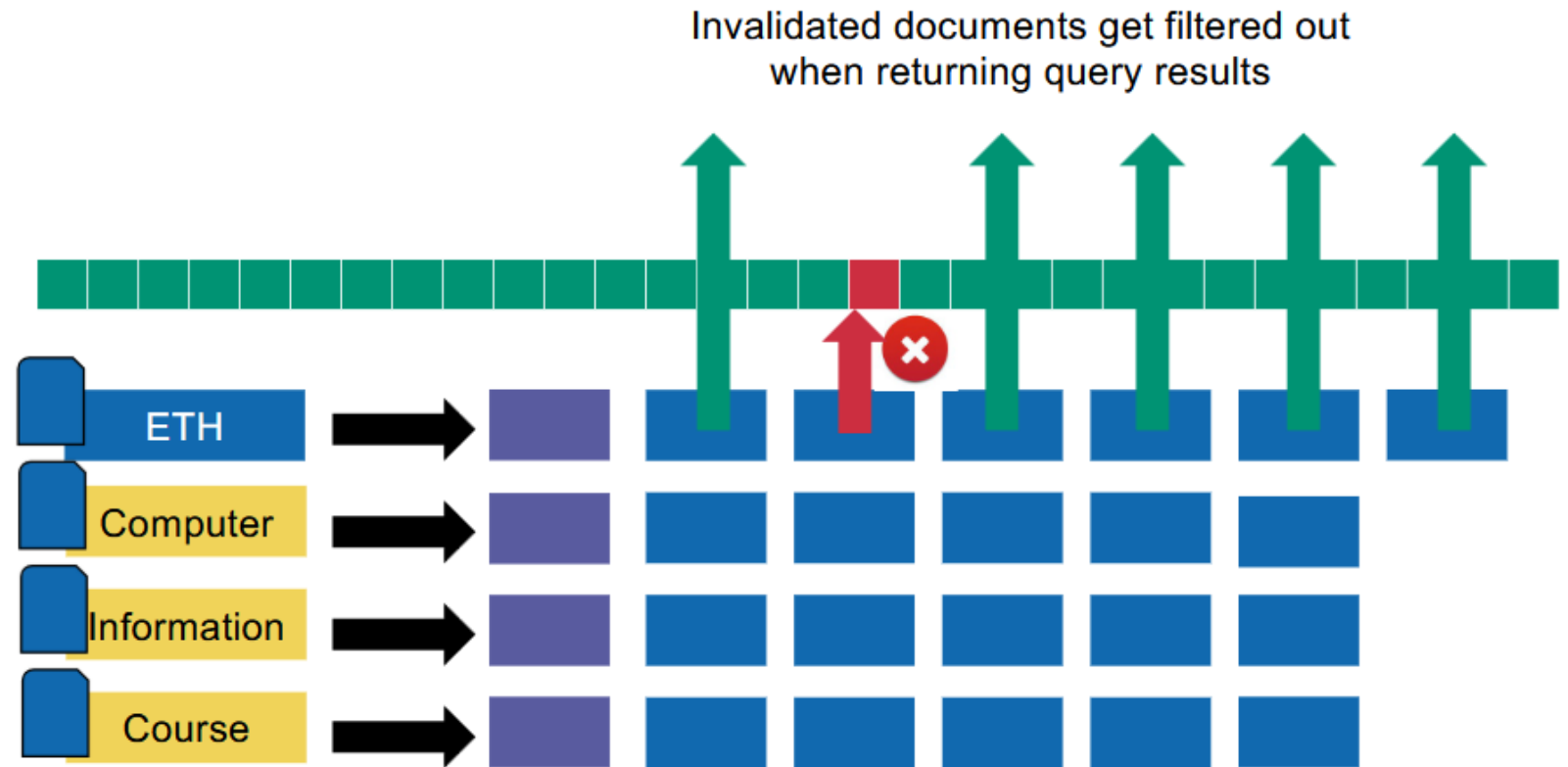
Auxiliary index

Store new documents in memory. Merge when memory is full.



Auxiliary index - Deletion

Use invalid bit



Optimization: Logarithmic merging

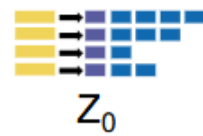
Normal merging: $O(\frac{T^2}{n})$

Logarithmic merging: $O\left(T \log\left(\frac{T}{n}\right)\right)$

n : size of auxiliary index

T : total number of postings

n postings



Z_0

$2n$ postings



Z_1

$4n$ postings



I_2



Exercise 4: Index Construction

- Implement the BSBI algorithm
 - *bsbi_invert()*
 - *merge_blocks()*
- Moodle Questions about BSBI/SPIMI, logarithmic merging

Exam Questions

While constructing the standard inverted index by processing the collection in sequential batches with the BSBI method, we encounter the following first batch of documents (in this order):

Document A

blue sky

Document B

blue water

Document C

cloudy sky

Document D

fresh water

Assign each term with its termID:

blue:

1

Exam Questions

While constructing the standard inverted index by processing the collection in sequential batches with the BSBI method, we encounter the following first batch of documents (in this order):

Document A

blue sky

Document B

blue water

Document C

cloudy sky

Document D

fresh water

Assign each term with its termID:

water:

3

Exam Questions

While constructing the standard inverted index by processing the collection in sequential batches with the BSBI method, we encounter the following first batch of documents (in this order):

Document A

blue sky

Document B

blue water

Document C

cloudy sky

Document D

fresh water

Assign each term with its termID:

fresh:

5

Exam Questions

While constructing the standard inverted index by processing the collection in sequential batches with the BSBI method, we encounter the following first batch of documents (in this order):

Document A

blue sky

Document B

blue water

Document C

cloudy sky

Document D

fresh water

Assign each term with its termID:

cloudy:

4

Exam Questions

Consider a collection of documents that get updated constantly. We use logarithmic merging to maintain the inverted index. Consider the current state (where I_4 is double I_3 is double I_2 is double I_1 is double I_0)

I_0, I_0, I_1, I_2, I_4

What is the first, second and final step in the cascade of merges?

I_1, I_1, I_2, I_4

I_2, I_2, I_4

I_3, I_4

Exam Questions

Mark T/F:

- ☐ Since computing the term frequency in a document is a sequential task, we cannot use MapReduce for it.
- ☐ In MapReduce, the mapping slots can interact with each other, while the reducer slots cannot.
- ☐ For the standard inverted index to be built successfully, we need each map task to successfully process its assigned batch of the collection. If this is not the case, we need to relaunch the failed map tasks.
- ☐ The reduction in MapReduce can only be done as a single reduce task, making it a bottleneck.
- ☐ Up to a certain point, using MapReduce for index constructions scales linearly with the number of machines.

Exam Questions

Mark T/F:

- ☐ Since computing the term frequency in a document is a sequential task, we cannot use MapReduce for it.
 - False, e.g. each mapper can count occurrences of each term, reducer can add up results
- ☐ In MapReduce, the mapping slots can interact with each other, while the reducer slots cannot.
 - False, mappings are not supposed to be affected by entries other than their inputs, while reducers are meant to combine results from multiple mappers
- ✓ For the standard inverted index to be built successfully, we need each map task to successfully process its assigned batch of the collection. If this is not the case, we need to relaunch the failed map tasks.
 - True, map tasks are independent, so you only need to relaunch failed tasks
- ☐ The reduction in MapReduce can only be done as a single reduce task, making it a bottleneck.
 - False, several reducers can work in parallel
- ✓ Up to a certain point, using MapReduce for index constructions scales linearly with the number of machines.
 - True, assuming no overhead for the distribution

Exam Questions

Mark T/F:

- ☐ The first step of BSBI, where we process each block separately and write back intermediate results at the end has complexity $O(T \log T)$, where $T = \text{\#tokens}$
- ☐ It is not possible to build a standard inverted index on a single machine if the corpus to index does not entirely fit in memory. A distributed system such as MapReduce must then be used.
- ☐ The overall complexity of SPIMI is $O(T)$
- ☐ The final merging phase of BSBI and SPIMI, in which the final, sorted, standard inverted index is written back to disk, has a complexity of $O(T)$ where $T = \text{\#tokens}$

Exam Questions

Mark T/F:

- ✓ The first step of BSBI, where we process each block separately and write back intermediate results at the end has complexity $O(T \log T)$, where $T = \text{\#tokens}$
 - Recall earlier today
- ❑ It is not possible to build a standard inverted index on a single machine if the corpus to index does not entirely fit in memory. A distributed system such as MapReduce must then be used.
 - False, the whole point of BSBI and SPIMI is to tackle corpuses (corpi?) larger than memory
- ✓ The overall complexity of SPIMI is $O(T)$
 - True, its $O(T \log M)$ and since $M \ll T$, the $\log M$ is negligible
- ✓ The final merging phase of BSBI and SPIMI, in which the final, sorted, standard inverted index is written back to disk, has a complexity of $O(T)$ where $T = \text{\#tokens}$
 - True, recall earlier today

Kahoot

<https://create.kahoot.it/share/information-retrieval-index-construction/e54661b7-58c4-472d-9ee1-62cbded680f1>